

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

深度优先搜索（DFS）从递归深入到回溯

Depth-First Search and Backtracking

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

1. 搜索与 DFS 概述
2. 递归与深入
3. 回溯：第一次接触
4. 图上的 DFS
5. 经典应用：回溯实战
6. 总结

目录

1 搜索与 DFS 概述
什么是搜索
2 递归与深入
3 回溯：第一次接触
4 图上的 DFS
5 经典应用：回溯实战

目录

6 总结

搜索：从一个起点找到答案

搜索：从初始状态出发，按照某种规则一步步扩展，直到找到目标状态。

- **状态**：当前的局面，比如“站在节点 3”
- **转移**：从一个状态走到另一个状态，比如“$3 \to 5$”
- **目标**：判断是否结束的条件，比如“到达节点 7”

搜索 = 系统地枚举所有可能走法 + 及时停掉没用的分支

两种基本搜索策略

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

DFS（深度优先）

- 一条路走到黑
- 靠递归 / 栈实现
- 先深入，再回头

BFS（广度优先）

- 一层一层往外扩
- 靠队列实现
- 先近后远

本次课只讲 DFS，BFS 留到后面

DFS 的直觉：走迷宫

把图想象成一座迷宫，你站在入口，目标是走到出口。

- 遇到岔路口，**随便挑一条**继续走
- 一路走到死胡同，就**原路退回**到最近的岔路口
- 换一条还没走过的路，继续往前走
- 走过的路做**标记**，不再重复走

“退回到岔路口”——这就是回溯的雏形

目录

1 搜索与 DFS 概述

2 递归与深入
递归回顾

3 回溯：第一次接触

4 图上的 DFS

5 经典应用：回溯实战

目录

6 总结

递归：函数调用自己

- **递归基**：什么时候停（不写会无限递归，栈溢出）
- **递归式**：问题怎么一步步变小
- **返回值**：结果怎么一层层传回来

```
1 int fact(int n) {
2     if (n <= 1) return 1;     // 递归基
3     return n * fact(n - 1);  // 递归式
4 }
```

递归调用栈：一层层压进去

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

`fact(4)` 的执行过程：

阶段	函数栈中的内容
深入	`fact(4)`
深入	`fact(4)` $\rightarrow$ `fact(3)`
深入	`fact(4)` $\rightarrow$ `fact(3)` $\rightarrow$ `fact(2)`
触底	`fact(4)` $\rightarrow$ `fact(3)` $\rightarrow$ `fact(2)` $\rightarrow$ `fact(1)`
返回	$1 \rightarrow 2 \rightarrow 6 \rightarrow 24$

函数栈是“后进先出”的：最后压进去的那一层，最先返回。

DFS 的递归骨架


```
void dfs(当前状态 u) {
    标记 u 已访问;                  // 1. 处理当前节点

    for (每个从 u 出发的邻居 v) {
        if (v 未访问) {
            dfs(v);                 // 2. 递归深入
        }
    }

    // 3. 如果需要“撤销”，就写在这里
}
```

第 3 步的“撤销”是可选的——有的问题需要，有的不需要，后面会讲清楚。

目录

1 搜索与 DFS 概述

2 递归与深入

3 回溯：第一次接触
什么是回溯

4 图上的 DFS

5 经典应用：回溯实战

目录

6 总结

先看一个问题：全排列

输出 $1 \sim 3$ 的所有排列，每个数只能用一次。

123 132 213 231 312 321

怎么才能**不重不漏**地全部枚举出来？

- 第 1 位：可以选 1、2、3
- 第 2 位：从剩下的数里选
- 第 3 位：只剩下一个数

选完之后必须“退回来”，才能去试别的组合。

什么是回溯

回溯：沿着一条路深入，发现走不通（或已经走完），就**退回上一步**，撤销刚才的选择，再试下一条路。

深入 → 碰壁 → 撤销 → 换路

- 回溯 = **DFS** + **状态恢复**
- DFS 负责“怎么往下走”
- 撤销负责“怎么退回来，并且把现场恢复原样”

回溯三步曲（本节最重要）

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

1. 做选择

- 记录当前选择
- 修改状态（标记、计数）

2. 递归

- 带着新状态进入下一层

3. 撤销选择

- 把状态改回原样
- 让这一步的选择“当作没发生”
- 这样才能干净地去试下一个选择

忘了第 3 步，答案会缺一大半

为什么必须撤销？

以“选数”为例，状态就是一个布尔数组 `used[1..3]`。

- 选了 1，把 `used[1]` 置为 `true`
- 再选 2，把 `used[2]` 置为 `true`，得到 1 2 3
- 接下来要试 1 3 2，就必须把 2 “放回去”
- 如果 `used[2]` 一直为真，那 2 就永远选不了了

撤销 = 把“我占用了这个数”改回“这个数空着”

回溯的通用模板

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
1  void backtrack(状态 state) {
2      if (到达终点) {
3          记录答案;
4          return;                    // 回溯点：返回上一层
5      }
6      for (每一种选择 c) {
7          if (!合法(c)) continue;
8          做出选择 c;                // 修改状态
9          backtrack(新状态);         // 递归深入
10         撤销选择 c;                // 恢复状态 *
11     }
12 }
```

带 * 的那一行，就是回溯的灵魂。

关键区分：什么时候要撤销？

问题类型	标记代表什么	要不要撤销
图的连通性	这个点“到过没有”	不撤销
连通块计数	同上	不撤销
枚举所有路径	当前路径占用谁	必须撤销
全排列 / 组合	当前方案选过谁	必须撤销

先问一句：这个标记是“全局到过”，还是“这一路占用”？

目录

1 搜索与 DFS 概述

2 递归与深入

3 回溯：第一次接触

4 图上的 DFS

邻接表与 visited

5 经典应用：回溯实战

目录

6 总结

图的存储回顾：邻接表

```cpp
const int N = 100005;
vector<int> adj[N];      // adj[u] 存 u 的所有邻居

// 加一条无向边 u-v
adj[u].push_back(v);
adj[v].push_back(u);
```

- **空间**：$O(V+E)$
- **遍历 u 的所有邻居**：$O(\deg(u))$
- **整张图完整遍历一遍**：$O(V+E)$

visited 数组：为什么必须有它？

图里可能有**环**，比如 $1 \to 2 \to 3 \to 1$。

- 没有 `visited`：$1 \to 2 \to 3 \to 1 \to 2 \to \cdots$ 无限递归
- 有 `visited`：每个点最多被访问一次，递归一定会停

`visited` 既是“去重”，也是“递归终止的保证”

注意：图的 DFS 结束，不是靠某条路走完，而是靠所有可达点都被访问过。

图上 DFS：完整代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
bool visited[N];

void dfs(int u) {
    visited[u] = true;          // 标记：来过了
    cout << u << " ";           // 处理当前点

    for (int v : adj[u]) {      // 枚举所有邻居
        if (!visited[v]) {
            dfs(v);             // 深入
        }
    }
}
```

注意：这里没有撤销 `visited`——因为“来过”是永久事实。

手算演练：写出 DFS 序

图（无向）：1-2，1-3，2-4，2-5，3-6，4-5，从 1 出发，邻居按编号从小到大访问。

步骤	说明
1	访问 1，邻居是 2、3
2	从 1 深入 2（邻居 1、4、5）
3	1 已访问，从 2 深入 4
4	2 已访问，从 4 深入 5
5	5 的邻居 2、4 都已访问 → 回溯
6	依次退回 4、2，都没有新邻居 → 回到 1
7	从 1 深入 3，再从 3 深入 6
8	6 没有新邻居，全部访问完毕

DFS 序：1 2 4 5 3 6

应用：数连通块

遍历所有点，每发现一个没访问过的点，就说明找到了一个新的连通块。

```
1 int cnt = 0;
2 for (int i = 1; i <= n; i++) {
3     if (!visited[i]) {
4         dfs(i);      // 从 i 出发，把整个连通块标记完
5         cnt++;
6     }
7 }
```

- 每次 dfs(i) 会“吞掉”一整个连通块
- 循环结束时 cnt 就是连通块个数
- 总复杂度仍然是 $O(V+E)$

目录

1 搜索与 DFS 概述

2 递归与深入

3 回溯：第一次接触

4 图上的 DFS

5 经典应用：回溯实战

全排列

网格上的路径搜索

目录

6 总结

应用一：全排列

求 $1 \sim n$ 的所有排列，每个数只能用一次。

- 状态：已经填好的前缀 `a[1..pos-1]`
- 选择：第 `pos` 位填哪个还没用过的数
- 终点：`pos > n`，此时前缀就是一个完整排列
- 撤销：把 `used[x]` 改回 `false`

“选一个 $\rightarrow$ 递归 $\rightarrow$ 放回去”，循环 n 次

全排列：代码实现


```
int n, a[15];
bool used[15];

void dfs(int pos) {
    if (pos > n) {                          // 已经填满了
        for (int i = 1; i <= n; i++) cout << a[i] << " ";
        cout << "\n";
        return;
    }
    for (int x = 1; x <= n; x++) {
        if (used[x]) continue;              // 这个数已经用过
        used[x] = true;                     // 做选择
        a[pos] = x;
        dfs(pos + 1);                       // 递归
        used[x] = false;                    // 撤销选择 *
    }
}
```

执行追踪：$n = 3$

把递归想象成一棵**选择树**，每条从根到叶的路径就是一个排列。

走到哪	状态	动作
第 1 层	选 1	标记 used[1]，深入
第 2 层	选 2	标记 used[2]，深入
第 3 层	选 3	输出 1 2 3，返回
回到第 2 层	撤销 3	used[3] 复位，没别的选择，继续返回
回到第 1 层	撤销 2	used[2] 复位，改选 3，继续搜

撤销之后，状态回到“这一层刚开始”的样子，才能干净地试下一个选择。

反面教材：不撤销会怎样？

如果去掉 `used[x] = false` 这一行：

- 第 1 层选了 1，`used[1]` 就永远为真
- 第 2 层选了 2，`used[2]` 也永远为真
- 第 3 层只剩 3，输出 1 2 3 后返回
- 回到第 2 层：1、2、3 全被标记，循环直接结束
- 回到第 1 层：同样全被标记，程序结束

结果只输出了 1 个排列，而不是 6 个

应用二：网格上的路径搜索

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

在 $n \times m$ 的网格里，# 是墙，. 是路，E 是终点。

```
int dx[4] = {1, -1, 0, 0};
int dy[4] = {0, 0, 1, -1};

bool dfs(int x, int y) {
    if (g[x][y] == 'E') return true;       // 到达终点
    vis[x][y] = true;
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
        if (g[nx][ny] == '#' || vis[nx][ny]) continue;
        if (dfs(nx, ny)) return true;      // 找到就一路返回 true
    }
    return false;
}
```

这里为什么不用撤销？

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

- 问题只问“**能不能到**”，不问“怎么走”
- 一个格子只要确认“到过”就够了，不需要重复走
- 所以 `vis` 是**全局**的，不撤销

但如果要输出所有路径呢？

- 同一个格子可以出现在不同的路径里
- 必须在函数末尾加上 `vis[x][y] = false`
- 否则第一条路径走过的格子，会把后面的路全堵死

判断“可达”不撤销；枚举“方案”要撤销

目录

1 搜索与 DFS 概述

2 递归与深入

3 回溯：第一次接触

4 图上的 DFS

5 经典应用：回溯实战

6 总结

目录

要点回顾

核心结论

- DFS 的核心是**递归深入**：一条路走到头，再回头
- 回溯 = DFS + **状态恢复**，三步是：做选择 → 递归 → 撤销
- 撤销的目的：让状态回到本层开始时的样子，好去试下一个选择
- 图搜索必须加 `visited`，否则有环时会无限递归
- 标记代表“全局到过”就不撤销；代表“当前路径占用”就必须撤销

递归负责“往下走”，撤销负责“走回来还能重新选”

速查表

场景	标记	撤销
图的连通性 / 连通块	visited	不撤销
网格可达性判断	vis	不撤销
枚举所有路径	vis	撤销
全排列 / 子集 / 组合	used	撤销
拓扑排序	visited	不撤销

先问自己：这个标记是“全局”的，还是“这一路”的？

附录：常见陷阱

- **忘记 `visited`**：有环图会无限递归，直接爆栈
- **忘记撤销**：答案会少一大半（全排列只剩 1 个）
- **撤销写错位置**：必须写在递归调用之后、且在同一次循环体内
- **递归太深**：链状图上 10^5 层可能爆栈，改用非递归写法
- **误以为 DFS 能求最短路**：无权图最短路要用 BFS

附录：DFS 的非递归写法（栈）

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
stack<int> st;
st.push(s);
while (!st.empty()) {
    int u = st.top(); st.pop();
    if (visited[u]) continue;
    visited[u] = true;
    cout << u << " ";
    for (int v : adj[u])
        if (!visited[v]) st.push(v);
}
```

用显式的 `stack` 模拟函数调用栈，可以避免递归爆栈。

附录：复杂度分析

场景	时间	空间
图 DFS（邻接表）	$O(V+E)$	$O(V)$
图 DFS（邻接矩阵）	$O(V^2)$	$O(V)$
网格搜索	$O(nm)$	$O(nm)$
全排列	$O(n \cdot n!)$	$O(n)$

全排列的答案本身就有 $n!$ 个，每个长度是 n，所以输出至少要 $O(n \cdot n!)$。